

An Overview of Programming Paradigms & Web Languages

Programming Paradigms

-A pattern or model for how the core language runs and interacts with individual elements of the program (functions, classes, variables, etc.)

I) Assembly - early programming, executed 100% sequentially, top to bottom

II) Functional - Main method of computation is applying arguments to functions. Functions treated as values where function is given a type, then a set of possible patterns where the pattern that matches the argument passed is executed. In pure functional, no external state (ex. *int i*;) outside of functions and side effects vs imperative lang.

-Ex. in Haskell

```
sum :: [int] --> int           //type
sum[n] = n
sum (n, ns) = n + sum ns      //patterns
//sum function takes in an array. If only one element in array, sum is n
//if multiple elements array, sum is n + sum of n's (via recursive call to sum)
```

III) Imperative - Main method of computation is assignment of values to variables.

-Ex. in Java

```
int total = 0;
for (int i = 0; i < 10; i++)
    total += i;
}
//for loop run and on each iteration updates the value of total by adding i to it
```

IV) Procedural - A type of imperative, where program can have more than one procedure (i.e. function). Made imperative more modular, but allowing subroutines. Top-down language. Ex. Python, PHP, COBOL.

V) Object Oriented - Allows encapsulation, where functions and properties are contained within separate objects, and objects interact with each other via public interfaces, which can access public/protected methods of an object. Allows for classes where a class is a blueprint for an object, which can be modified or built on via inheritance. Allows for easy code re-use via top-down procedural. Allows for SOLID and other best practices. Ex. Python, PHP, Java.

VI) Event-driven - Flow of program is driven by events (ex. mouse click) where there is typically a main loop that listens for events, then when an event occurs, the proper function is executed to handle that event (the event handler). Core type of programming for GUI.

Other Types of Languages

I) Markup Language - A way to annotate a document where the plaintext (ex. paragraph) is syntactically distinguishable from instructions used to give that text properties, such as appearance details. Ex. HTML.

II) Style Sheet - Expresses how to define a structured document such as an HTML doc via stylistic

rules applied to document elements via selectors, which choose elements, and via rulesets, which apply values to properties of elements. Ex. CSS stylesheet.

Types of Execution

I) Interpreted - code is directly executed by the interpreter, which is a program that performs the instructions written in the code, without requiring the code to be previously compiled into machine language. Ex. Python, PHP, Perl, Ruby

II) Compiled - prior to execution of program, code must be run through compiler, which translates source code into machine code. Could run compiled code as native code, or as bytecode and executed through means such as Java virtual machine or C# CLR. Example: C, C++, C#, Java, Go, Lisp

Characteristics of Languages

I) Dynamic - aka dynamically typed. Execute common programming behaviors that static languages would perform during compilation. Example: adding new code, extending objects, changing the type of a variable, etc. Objects can be given different types values.

Python Example:

```
employeeName = 9
employeeName = "Steve Ferg"    //no error
```

II) Static - Once an object is bound to a type, the type of the object cannot be changed.

Java Example:

```
int employeeName = 9;
empolyeeName = "Stever Ferg";    //Java throws error as String cannot bind to int
```

III) Weakly typed - language will implicitly (automatically) try and coerce variable into unrelated type if variable is being used in situation where current type is not accepted. Example. Perl, JavaScript.

Javascript Example:

```
let a = 9;
let b = "9";
let c = b.concat(a);    //produces "99"
let d = sum(a, b);    //produces 18
```

IV) Strongly typed - language will not implicitly coerce variable into unrelated type if variable is being used in situation where current type is not accepted. Example. Ruby, Java, C#

Types of Structured Programming

-Structured - considered a precursor to OOP, where OOP is considered a type of structured, where program is divided into multiple sections with multiple exits and one access point. Comprised as control structures, such as iterations, conditional statements, subroutines, etc.

I) Class-based - A clear distinction exists between the class, which is the blueprint interface defining the method and properties of an object, and an instance of the class (an object). The class exists independently from the objects, which define their type of object. Objects can override methods of classes. Classes can inherit from each other, and objects created reflect this. Example: Java, C++, C#, Python, PHP

II) Prototype-based - There is no concept of classes and instances. Instead, there are objects. Properties are given by specification specific to that object and functions can be added to a class during runtime.

JavaScript example:

```
let obj = { name: "John" } //object created
obj.sayName = function() { console.log("My name is + " this.name); //add funct to obj
obj.sayName();           //call newly added function
```

-Inheritance is done by using an object as a prototype, where that object is used as a template for other objects, then objects are created as clones of the prototype. Clones can override inherited methods and this will not affect the prototype. If prototype changes, clones created from it will be affected. JS works like this.

Web Languages (and Libraries/Frameworks)

HTML

-Hypertext Markup Language

-The most basic component of websites and places things like links, images, paragraphs, headers, lists, etc. on the site

-A markup language, not a programming language, so does not have functionalities like loops, conditional statements, etc.

-Based around elements, where an element is an item on your page (ex. an image) and attributes, where an attribute tells a property about your element (ex. title that shows as text bubble when image hovered over)

-Ex.

```
<a href="lainchain.org" title="Lainchan">Visit Lainchan</a>
//turns "Visit Lainchan" into a hyperlink
```

CSS

-Cascading Stylesheets

-Used to change the appearance of the site, such as set font size, change colors, arrange HTML elements to a specific layout, set backgrounds, give elements borders, align items, etc. Layout displays such as *grid* and *flex* allow fluid, responsive layout for elements. Media queries allow you to apply different rules in different scenarios (ex. on smaller screens, such as cell screens, versus larger screens)

-Based around selectors and rules, where rules define how HTML should be styled and where the selector selects which element(s) should be styled

-Example:

```
.imageBox {
  background-color: black;
  border: solid black 3px;
  color: yellow;
  font-size: 16px;
  text-align: center;
}
// applies the state rules to all HTML elements of class imageBox
```

JavaScript

- An interpreted, weakly typed dynamic language
- Allows for dynamic sites, meaning sites update according to user interaction with, vs static site, which is in an unchanging state.

- Multi-paradigm: event-driven, functional, imperative (OOP and prototype-based)

- Prototype-based, meaning inheritance is done by re-using existing objects where objects are extended by cloning an object and would have greater detail added to it. Each instance of x object is a clone object. Differs from class based, where inheriting objects transitively extend from a class.

- Very common in websites, with browsers containing JavaScript engine which is used to execute JS code

- Includes core API for array, date, regex, dom manipulation, etc functionalities, but does not include any core I/O, which it relies on host environment for

- Example:

```
function arrayCompute(array, computationFunction){  
    for(element of array)  
        console.log(computationFunction(element));  
}  
//takes an array and a function and prints the outcome of the function run on each array element
```

Python

- An interpreted, high-level, general-purpose back-end programming language

- Dynamically typed, garbage-collected, strongly typed language

- Multi-paradigm: supports procedural, object-oriented, and functional programming

- Emphasizes code readability with significant whitespace and a clean, simple syntax, which is intended to allow programmers to express concepts in fewer lines of code

- Uses indentation for code blocks rather than curly braces or keywords like "begin" and "end"

- Includes extensive standard library, often described as having "batteries included" philosophy, providing tools for many common programming tasks

- Example:

```
def greet(name):  
    return f"Hello, {name}!"  
  
names = ["Alice", "Bob", "Charlie"]  
for name in names:  
    print(greet(name))  
// define function to print names, then names, then print all names
```

- JavaScript front-end requests to Python back-end to PUT, POST, etc. or to GET, etc.

JSON

-JavaScript Object Notation

-A lightweight data-interchange format, easy for humans to read and write, and easy for machines to parse and generate

-Language-independent text format that uses conventions familiar to programmers of C-family languages

-Built on two structures: a collection of key/value pairs (object) and an ordered list of values (array)

-Primarily used for transmitting data between a server and web application, as an alternative to XML

-Example:

```
{
  "team": "Billing",
  "numEmployees": 3,
  "employees": [
    {"firstName": "John", "lastName": "Doe", "age": 28},
    {"firstName": "Anna", "lastName": "Smith", "age": 32},
    {"firstName": "Peter", "lastName": "Jones", "age": 40}
  ]
}
// defines a JSON object to detail out the Billing team
```

AJAX

-Asynchronous JavaScript And XML

-A technique for creating fast, dynamic web pages

-Allows web pages to update asynchronously by exchanging small amounts of data with the server behind the scenes, meaning parts of a web page can be updated without reloading the whole page

-Despite the name, XML is not required (JSON is often used instead), and the requests don't need to be asynchronous

-Core components include the *XMLHttpRequest* object (or more modern *Fetch* API) and DOM manipulation

-Example:

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'data.json', true);

xhr.onload = function() {
  if (this.status === 200) {
    const response = JSON.parse(this.responseText);
    document.getElementById('output').textContent = response.message;
  }
};
```

```
xhr.send();  
//requests data.json then updates an element with the response
```

Fetch

- A modern API that provides an interface for fetching resources (including across the network)
- Provides a more powerful and flexible feature set than XMLHttpRequest, including request/response streams and better promise-based syntax
- Returns promises that resolve to the Response object representing the response to the request
- More straightforward to use with promise chaining or async/await syntax than older AJAX methods

-Example:

```
fetch('https://api.example.com/data')  
  .then(response => {  
    if (!response.ok) {  
      throw new Error('Network response was not ok');  
    }  
    return response.json();  
  })  
  .then(data => console.log(data))  
  .catch(error => console.error('Fetch error:', error));  
//fetches data from API and processes the JSON response
```

Axios

- A promise-based HTTP client for the browser and back-end communication
- Makes it easy to send asynchronous HTTP requests to REST endpoints and perform CRUD operations
- Provides a more powerful and flexible feature set than native fetch, including automatic JSON transformation, request/response interception, and better error handling
- Simplifies handling of complex authentication schemes and has built-in protection against XSRF attacks

-Example:

```
axios.get('https://api.example.com/users')  
  .then(response => {  
    console.log(response.data);  
  }).catch(error => {  
    console.error('Error fetching data:', error);  
  });  
// basic GET and a configured POST request
```

SASS

-Syntactically Awesome Style Sheets

-A pre-processor scripting language that is interpreted or compiled into CSS

-Adds programming features to CSS like variables, nesting, mixins, inheritance, and other features not available in standard CSS

-Two syntax available: SCSS (Sassy CSS) which uses CSS-like syntax with curly braces, and indented syntax which uses indentation instead of braces

-Needs to be compiled to standard CSS before being served to the browser

-Example:

```
$primary-color: #3498db;          //variable  
$padding: 10px;
```

```
.container {  
  background-color: lighten($primary-color, 20%);    //SASS function call  
  padding: $padding;
```

```
  .button {  
    background-color: $primary-color;  
    border: none;  
    padding: $padding;
```

```
    &:hover {  
      background-color: darken($primary-color, 10%);    //nesting  
    }  
  }  
}
```

Bootstrap

-A free, open-source CSS framework directed at responsive, mobile-first front-end web development

-Contains CSS and JavaScript-based design templates for typography, forms, buttons, navigation, and other interface components

-Grid system that uses a series of containers, rows, and columns to layout and align content, which makes it easy to design responsive websites

-Includes pre-built components like navigation bars, buttons, forms, cards, and modals that can be easily implemented

-Example:

```
<div class="container">  
  <div class="row">  
    <div class="col-md-6">  
      <div class="card">  
        <div class="card-body">
```

```

        <h5 class="card-title">Card Title</h5>
        <p class="card-text">Some quick example text.</p>
        <a href="#" class="btn btn-primary">Go somewhere</a>
    </div>
</div>
</div>
</div>
</div>
//creates a responsive card layout using Bootstrap classes

```

-Made much less often needed to to CSS 5 adding grid and flexbox layouts

React

-A JavaScript library for building user interfaces, maintained by Facebook and a community of developers

-Component-based architecture where UI is broken down into reusable, isolated components that manage their own state (great for functional UI and SPA)

-Uses a virtual DOM to optimize rendering performance by minimizing actual DOM operations

-Declarative approach that makes code more predictable and easier to debug

-Usually paired with JSX, an extension that allows writing HTML-like code within JavaScript

-Includes Hooks (introduced in React 16.8) which allow managing state and other React features in functional components without classes

-Common hooks include *useState* for cross-component state management, *useEffect* for side effects, *useContext* for context consumption, and *useRef* for persistent mutable values

-Example:

```

function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}

```

```

function App() {
  return (
    <div>
      <Welcome name="Alice" />
      <Welcome name="Bob" />
      <Welcome name="Charlie" />
    </div>
  );
}

```

```

ReactDOM.render(<App />, document.getElementById('root'));

```


React Redux

-A predictable state container for React applications

-Implements a unidirectional data flow pattern where all application state is stored in a single centralized store

-Based on three core principles: single source of truth, state is read-only, and changes are made with pure functions (reducers)

-Uses actions (plain JS objects) to describe state changes and reducers to specify how the state changes in response to actions

-Provides middleware support for handling side effects, async operations, and complex flows

-Includes DevTools for time-travel debugging, state inspection, and action replay

-Example:

// Action types

```
const INCREMENT = 'INCREMENT';
```

```
const DECREMENT = 'DECREMENT';
```

// Action creators

```
function increment() {  
  return { type: INCREMENT };  
}
```

```
function decrement() {  
  return { type: DECREMENT };  
}
```

// Reducer

```
function counterReducer(state = { count: 0 }, action) {  
  switch (action.type) {  
    case INCREMENT:  
      return { count: state.count + 1 };  
    case DECREMENT:  
      return { count: state.count - 1 };  
    default:  
      return state;  
  }  
}
```

// Store

```
const store = Redux.createStore(counterReducer);
```

// React component with Redux

```
function Counter() {  
  return (  
    <div>
```

```

        <span>Count: {store.getState().count}</span>
        <button onClick={() => store.dispatch(increment())}>+</button>
        <button onClick={() => store.dispatch(decrement())}>-</button>
    </div>
);
}

```

Django

-A high-level Python web framework that follows the model-view-template (MVT) architectural pattern

-Emphasizes re-usability and "pluggability" of components, rapid development, and the principle of "Don't Repeat Yourself" (DRY)

-Includes a powerful Object-Relational Mapping (ORM) layer for database interactions without writing SQL

-Features built-in admin interface, authentication system, URL routing, template engine, and security features

-Example:

```

# models.py
class Article(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    pub_date = models.DateTimeField('date published')

# views.py
def article_detail(request, article_id):
    article = get_object_or_404(Article, pk=article_id)
    return render(request, 'articles/detail.html', {'article': article})

# urls.py
urlpatterns = [
    path('articles/<int:article_id>/', views.article_detail, name='article_detail'),
]

```

Flask

-A micro web framework for Python, designed to be simple and easy to extend

-Doesn't include database abstraction layer, form validation, or any other components where pre-existing "add your own as requirements specify" third-party libraries provide common functions

-Built around Werkzeug WSGI toolkit and Jinja2 template engine

-Considered more lightweight compared to Django, giving developers more flexibility but requiring more explicit configuration

-Example:

```

from flask import Flask, render_template

```

```
app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello, World!'

@app.route('/user/<username>')
def show_user_profile(username):
    return render_template('user.html', username=username)

if __name__ == '__main__':
    app.run(debug=True)
//creates a simple Flask application with two routes
```